Práctica 7: Gráficos en R

Laboratorio de Matemáticas

Grado en Ingeniería Matemática - Doble Grado en Física e Ingeniería Matemática

Miguel Beltra Vidal

Curso 2026/27

1. Objetivos

El objetivo de esta práctica es aprender a crear gráficos utilizando R. para representar información mediante gráficos. Al acabar la práctica, el estudiante podrá representar cualquier gráfico que necesite para sus trabajos usando R.

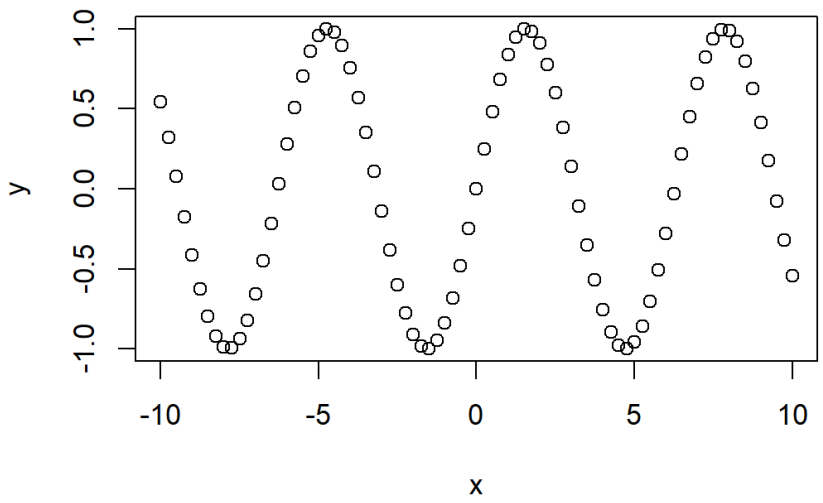
2. Primeros gráficos en R

2.1. Función `plot`

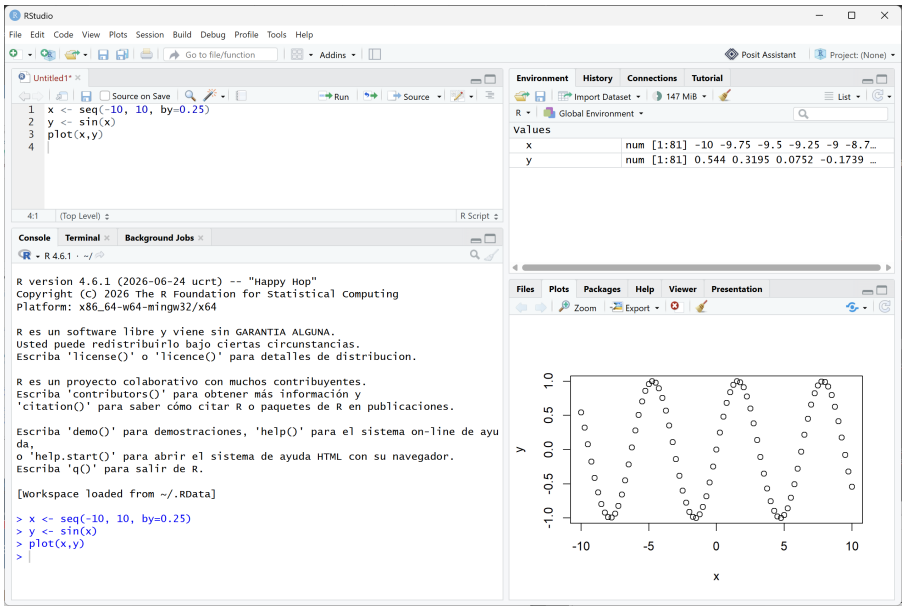
R permite crear gráficos directamente a partir de vectores. Una de las funciones fundamentales es `plot()`. A esta función tenemos que especificarle las coordenadas x y las coordenadas y de los puntos que queremos que dibuje. Por ejemplo, las siguientes instrucciones:

```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y)
```

tienen como salida:

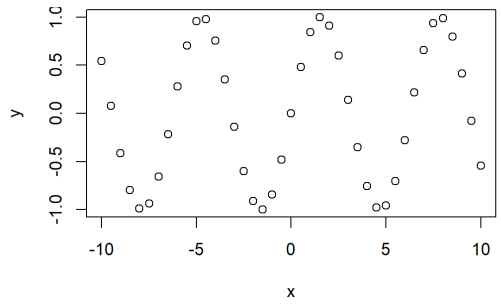


En RStudio los gráficos generados se mostrarán en la esquina inferior derecha. Desde esta misma ventana podemos exportar los gráficos que generemos desde el botón `Export`.

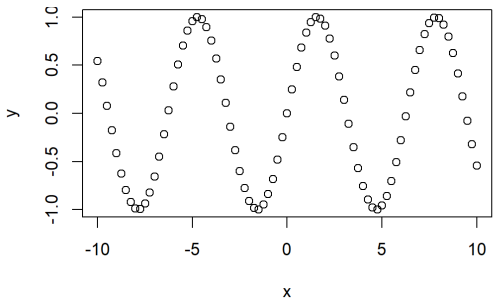


Podemos añadir más puntos para obtener mejores gráficas. Las siguientes gráficas se corresponden con las siguientes instrucciones:

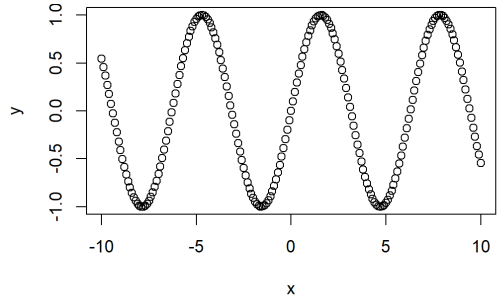
```r
x <- seq(-10, 10, by=0.5)
y <- sin(x)
plot(x,y)
```



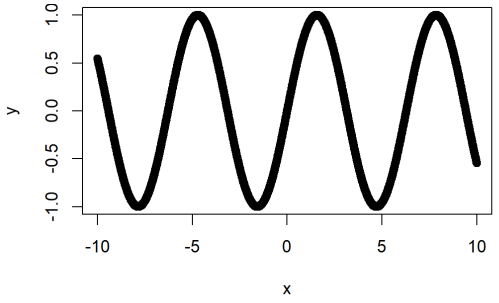

```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y)
```




```r
x <- seq(-10, 10, by=0.1)
y <- sin(x)
plot(x,y)
```




```r
x <- seq(-10, 10, by=0.01)
y <- sin(x)
plot(x,y)
```



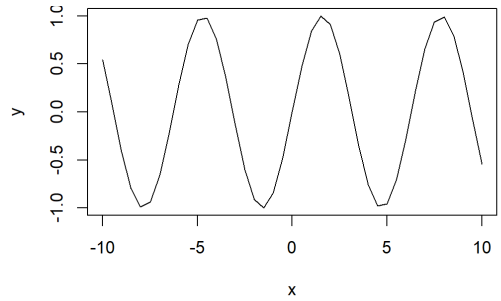
2.2. Argumento `type` - Tipo de representación

El argumento `type` permite modificar el tipo de representación. Sus posibles valores son:

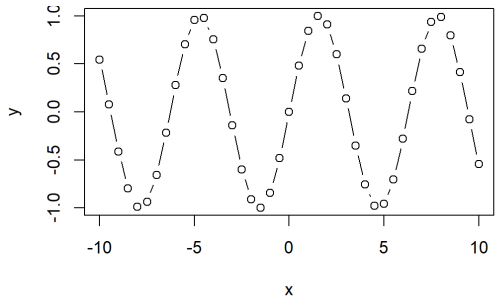
- `"p"`: para representación con puntos.
- `"l"`: para representación con líneas.
- `"b"`: para representación con ambos, esto es, puntos unidos mediante líneas.

El tipo `"p"` es el que se dibuja por defecto. Los otros tipos se muestran a continuación:

```
x <- seq(-10, 10, by=0.5)
y <- sin(x)
plot(x, y, type = "l")
```




```
x <- seq(-10, 10, by=0.5)
y <- sin(x)
plot(x, y, type = "b")
```



2.3. Argumento `col` - Color del gráfico

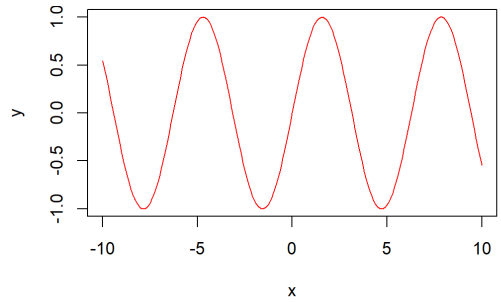
El argumento `col` permite modificar el color de la representación. Algunos de los valores más habituales son:

```
"white"   "black"    "blue"   "brown"   "cyan"   "darkgreen"   "gray"
"green"   "magenta"  "orange"  "pink"   "purple"    "red"      "yellow"
```

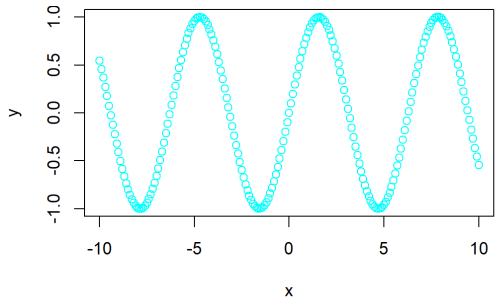
Podemos ver una lista completa de los colores predefinidos que podemos especificar mediante una cadena de texto con el comando:

```
> colors()
```

```
x <- seq(-10, 10, by=0.1)
y <- sin(x)
plot(x,y,type="l",col="red")
```

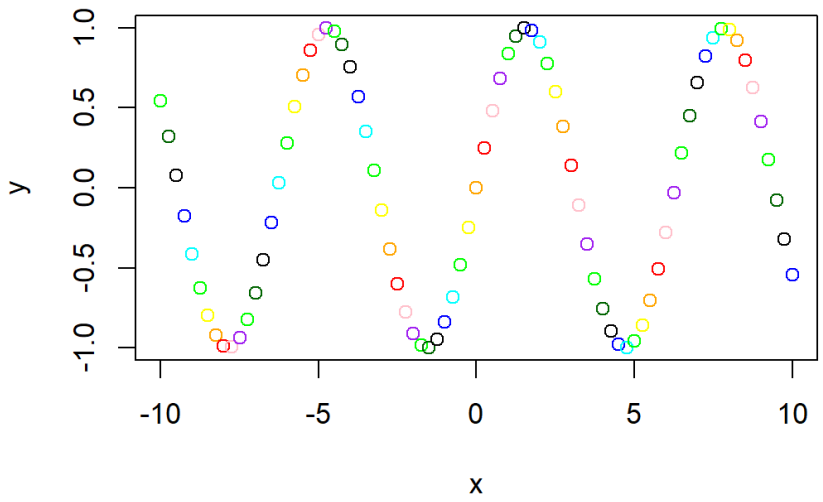



```
x <- seq(-10, 10, by=0.1)
y <- sin(x)
plot(x,y,type="p",col="cyan")
```



Si damos un vector de colores, entonces los puntos se colorearán de forma alternada de esos colores.

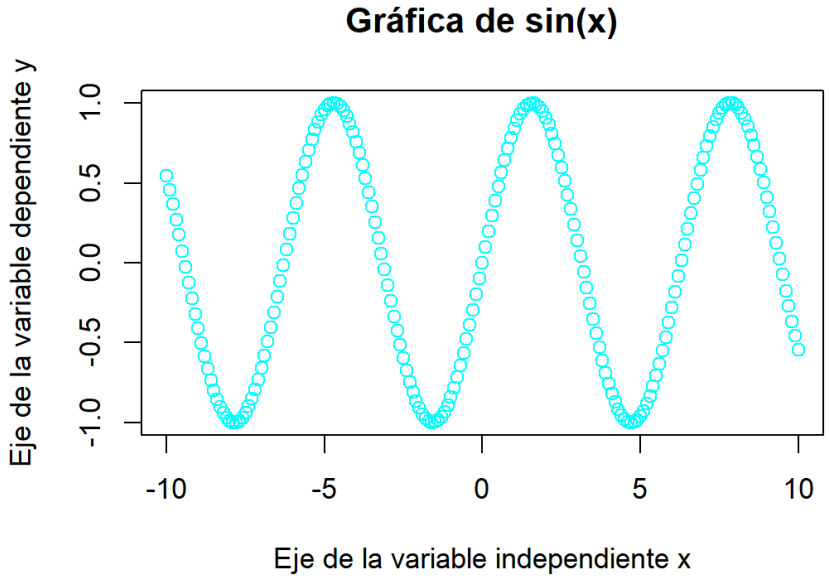
```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "p",
          col = c("green","darkgreen","black","blue","cyan","green",
                  "yellow","orange","red","pink","purple"))
```



2.4. Argumentos `main`, `xlab` y `ylab` - Título y etiquetas en los ejes

El argumento `main` permite poner título al gráfico. Los argumentos `xlab` e `ylab` permiten cambiar el nombre a los ejes.

```r
x <- seq(-10, 10, by=0.1)
y <- sin(x)
plot(x,y,type = "p",
          col = "cyan",
          main = "Gráfica de sin(x)",
          xlab = "Eje de la variable independiente x",
          ylab = "Eje de la variable dependiente y")
```



2.5. Argumento pch - Tipo de punto

El tipo de punto se puede especificar con el argumento `pch` (*plotting character*). Podemos asignarle un número entre el 1 y el 25. En ese caso, el tipo de punto viene dado por:

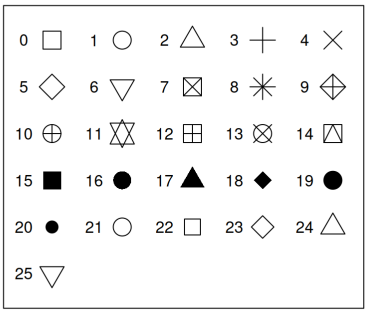
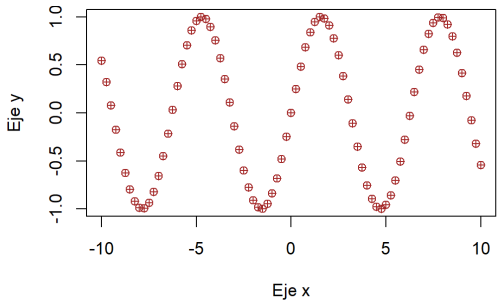


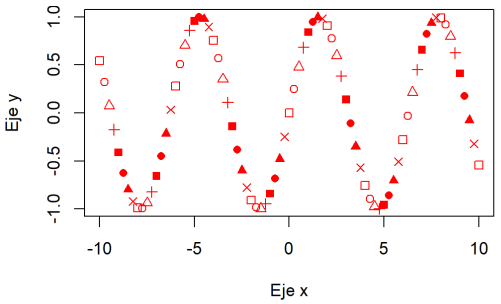
Figura 1: Imagen obtenida de `https://r-charts.com/base-r/pch-symbols/`

También podemos especificar una lista de valores para indicar que se alternan ciertos símbolos, o incluso podemos especificar una letra para que sea el símbolo del punto.

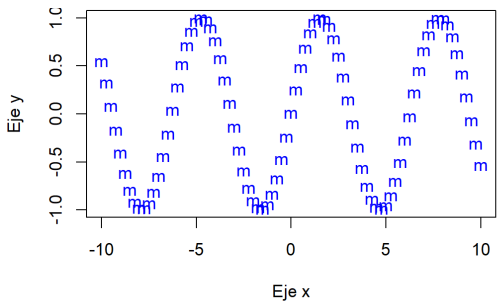
```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type="p",
  col="brown", main="sin(x)",
  xlab="Eje x", ylab="Eje y",
  pch=10)
```



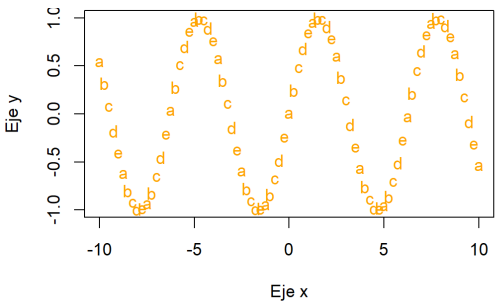

```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type="p",
  col="red", main="sin(x)",
  xlab="Eje x", ylab="Eje y",
  pch=c(0,1,2,3,15,16,17,4))
```




```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type="p",
  col="blue", main="sin(x)",
  xlab="Eje x", ylab="Eje y",
  pch="m")
```



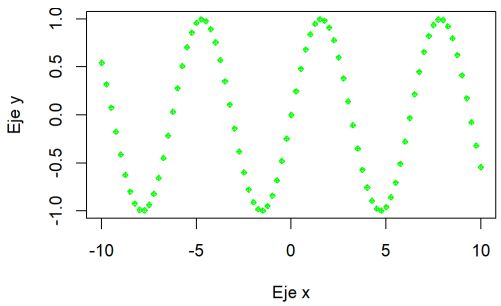

```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type="p",
  col="orange", main="sin(x)",
  xlab="Eje x", ylab="Eje y",
  pch=c("a","b","c","d","e"))
```



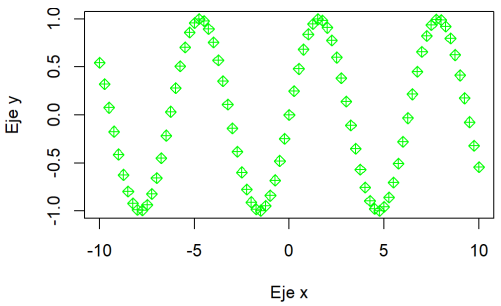
2.6. Argumento cex - Tamaño del punto

Con el argumento `cex` podemos especificar el tamaño del punto. Por defecto, este tamaño es 1. Si especificamos varios valores, entonces los puntos tomarán dichos valores.

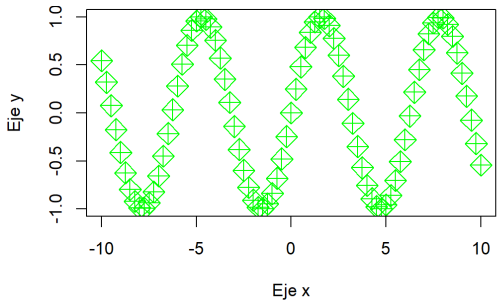
```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "p",
          col = "green",
          main = "sin(x)",
          xlab = "Eje x",
          ylab = "Eje y",
          pch = 9,
          cex = 0.5)
```



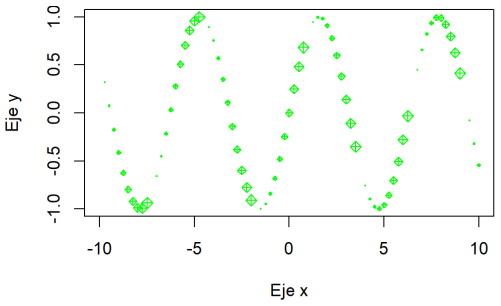

```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "p",
          col = "green",
          main = "sin(x)",
          xlab = "Eje x",
          ylab = "Eje y",
          pch = 9,
          cex = 1)
```




```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "p",
          col = "green",
          main = "sin(x)",
          xlab = "Eje x",
          ylab = "Eje y",
          pch = 9,
          cex = 2)
```




```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "p",
          col = "green",
          main = "sin(x)",
          xlab = "Eje x",
          ylab = "Eje y",
          pch = 9,
          cex = seq(0,1,by=0.1))
```



2.7. Argumento `lty` - Tipo de línea

Podemos especificar el tipo de línea con el argumento `lty` (*line type*).

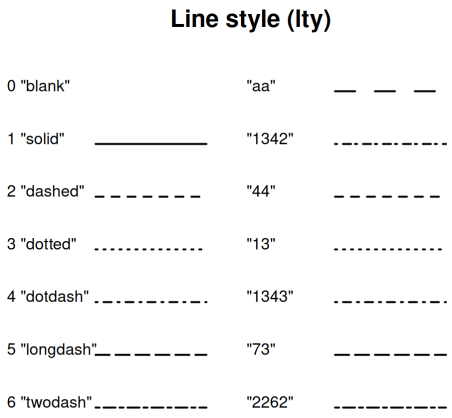
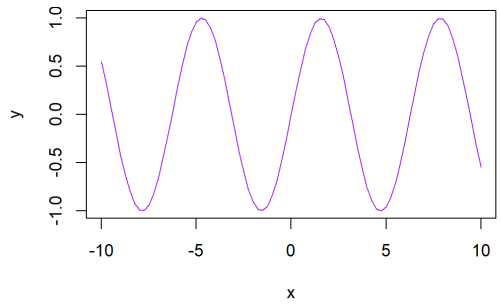
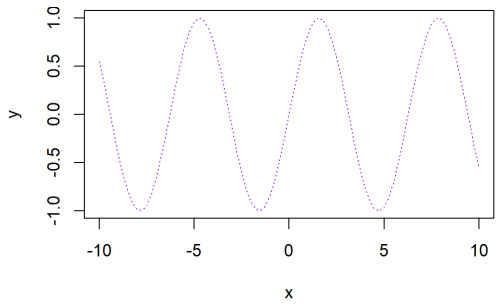


Figura 2: Imagen obtenida de `https://r-charts.com/base-r/line-types/`

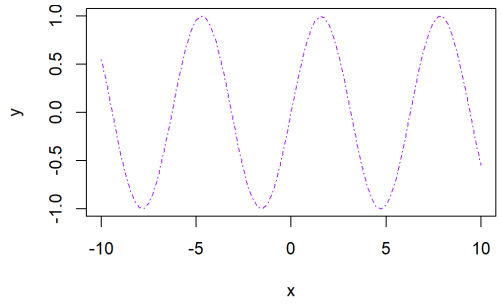
```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "purple",
          lty = 1)
```



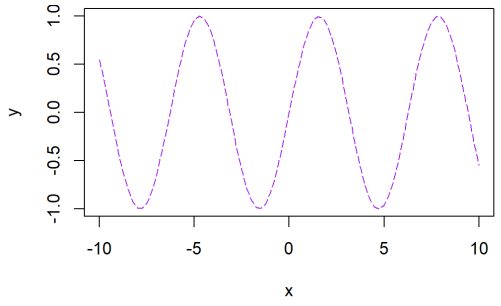

```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "purple",
          lty = 3)
```




```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "purple",
          lty = 4)
```



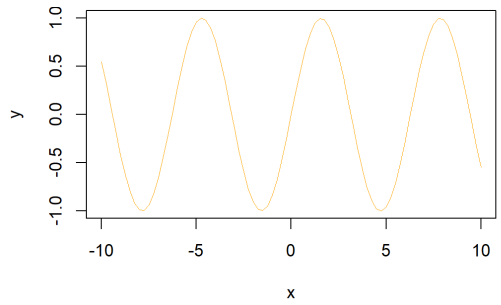

```r
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "purple",
          lty = 5)
```



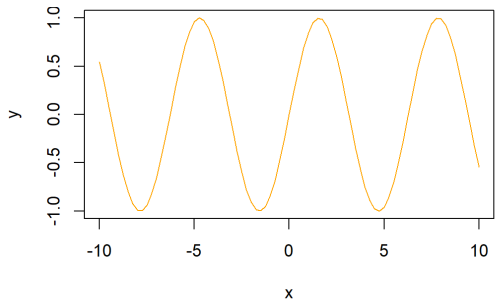
2.8. Argumento `lwd`

El argumento `lwd` (*line width*) define el grosor de la línea del `plot()`.

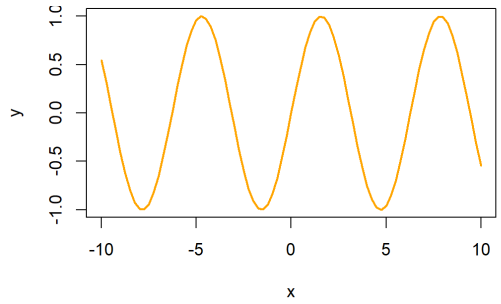
```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "orange",
          lwd = 0.5)
```



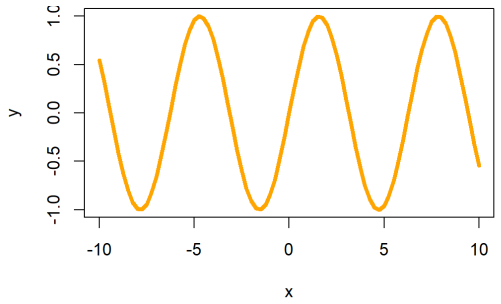

```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "orange",
          lwd = 1)
```




```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "orange",
          lwd = 2)
```



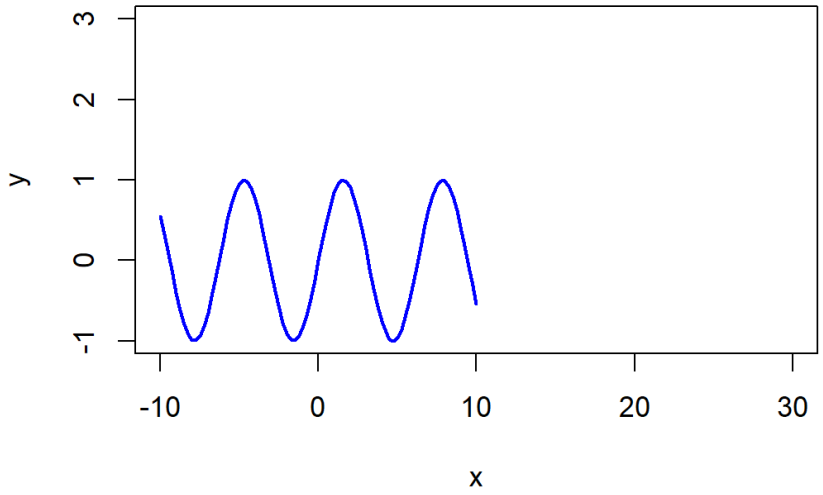

```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
          col = "orange",
          lwd = 4)
```



2.9. Argumentos `xlim` e `ylim`: límites de los ejes

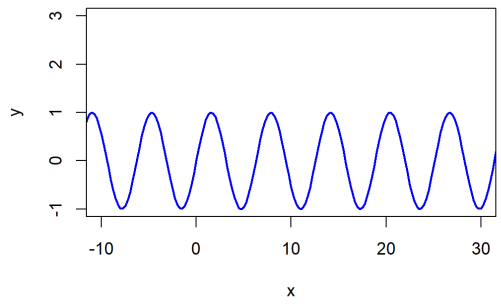
Los argumentos `xlim` e `ylim` nos permiten especificar los límites del gráfico. Se especifican con un vector de dos elementos: `c(lim_inf,lim_sup)`.

```
x <- seq(-10, 10, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
     col = "blue",
     lwd = 2,
     xlim = c(-10,30),
     ylim = c(-1,3))
```

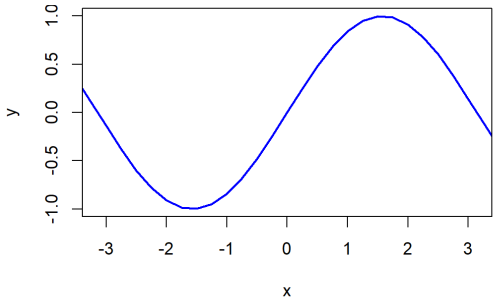


Observa que no se pintarán puntos que no se han especificado. Estos argumentos nos sirven para poder enfocar la región que nos interesa del gráfico.

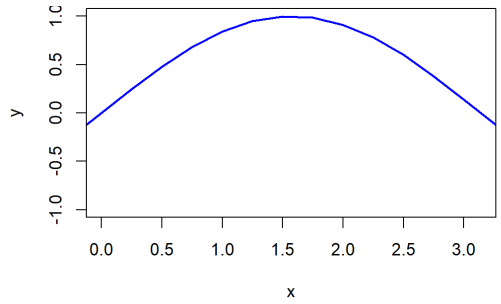
```
x <- seq(-35, 35, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
     col = "blue", lwd = 2,
     xlim = c(-10,30),
     ylim = c(-1,3))
```



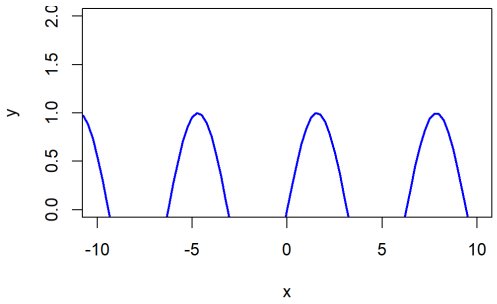

```
x <- seq(-35, 35, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
     col = "blue", lwd = 2,
     xlim = c(-pi,pi),
     ylim = c(-1,1))
```




```
x <- seq(-35, 35, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
     col = "blue", lwd = 2,
     xlim = c(0,pi),
     ylim = c(-1,1))
```




```
x <- seq(-35, 35, by=0.25)
y <- sin(x)
plot(x,y,type = "l",
     col = "blue", lwd = 2,
     xlim = c(-10,10),
     ylim = c(0,2))
```



Ejercicio 1. define las siguientes funciones matemáticas básicas:

Función	Descripción
`x^n`	Potencia n-ésima de x
`sqrt(x)`	Función raíz cuadrada
`exp(x)`	Función exponencial
`log(x)`	Función logaritmo neperiano
`log(x,n)`	Función logaritmo en base n
`log2(x)`	Función logaritmo en base 2
`log10(x)`	Función logaritmo en base 10
`sin(x)`	Función seno
`cos(x)`	Función coseno
`tan(x)`	Función tangente
`asin(x)`	Función arcoseno
`acos(x)`	Función arcocoseno
`atan(x)`	Función arcotangente
`sinh(x)`	Función seno hiperbólico
`cosh(x)`	Función coseno hiperbólico
`tanh(x)`	Función tangente hiperbólica
`asinh(x)`	Función arcoseno hiperbólico
`acosh(x)`	Función arcocoseno hiperbólico
`atanh(x)`	Función arcotangente hiperbólica
`sign(x)`	Función signo
`abs(x)`	Función valor absoluto
`trunc(x)`	Función parte entera
`round(x)`	Entero más cercano
`floor(x)`	Entero más cercano por defecto
`ceiling(x)`	Entero más cercano por exceso

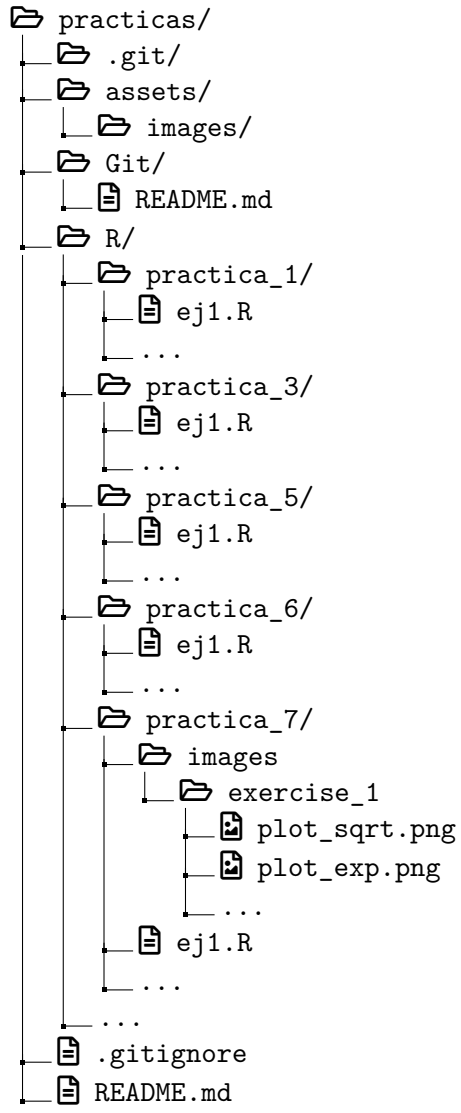
Para cada una de estas funciones, dibuja su gráfica. Deberás seguir las reglas siguientes:

1. En el caso de las potencias, deberás representar al menos los exponentes $n = 1, 2, 3, 4, 5, 6$.
2. En total son 30 gráficas: 10 de ellas se representarán con puntos, 10 con líneas y 10 con puntos y líneas.
3. Cada gráfica deberá ser de un color distinto (utiliza la función `colors()` para consultar los colores que tiene predefinidos). Además, al menos dos de ellas utilizarán varios colores alternados (al menos 4).
4. Todas las gráficas deberán tener un título indicando la función que se está representando. Además, se deberán añadir etiquetas en los ejes.
5. Cada gráfica que utilice puntos (incluidas las de líneas y puntos) deberá usar un tipo de punto distinto. Además, al menos dos de ellas utilizarán varios tipos de puntos de forma alternada: una utilizará tipos definidos por (al menos 4), y otra utilizará como puntos las letras de tu nombre.
6. De las 10 gráficas que utilizan puntos, 2 de ellas usarán un tamaño de punto pequeño, 2 un tamaño mediano, 2 un tamaño grande y 2 un tamaño muy grande. Además, 2 utilizarán varios tamaños de puntos de forma alternada.
7. Utiliza 5 tipos de línea distintos para las 10 gráficas que utilizan líneas, con 2 gráficas para cada grosor.

8. Utiliza 5 grosores distintos para las 10 gráficas que utilizan líneas, con 2 gráficas para cada grosor, con la restricción de que dos gráficas con el mismo tipo de línea no pueden tener el mismo grosor de línea.

9. Los límites de la gráfica deberán escogerse para que se pueda apreciar la forma de la gráfica: utiliza valores de `xlim` e `ylim` que consideres adecuados. Escoge también un rango de valores adecuado para la función.

Almacenaremos las gráficas que generemos en un directorio que llamaremos `exercise_1`, dentro de otro llamado `images`, que se encontrará en el directorio en el que estamos guardando los ficheros de la práctica 7. Utilizaremos nombres que sean descriptivos de la gráfica. Nuestro repositorio debería tener una forma similar a la siguiente siguiente al acabar el ejercicio.



3. Gráficas con multiples elementos

En R podemos combinar múltiples gráficos en uno con las funciones `plot()`, `lines()`, `points()` y `abline()`. Las reglas son las siguientes:

1. El gráfico inicial lo haremos con `plot()`.
2. Una vez creado el gráfico inicial, utilizaremos `lines()`, `points()` y `abline()`.
3. Cuando queramos empezar un nuevo gráfico, volveremos a utilizar `plot()`.

A medida que vamos ejecutando los comandos `lines()` y `points()`, vamos añadiendo nuevos elementos al gráfico. Todos los argumentos del apartado anterior para plot también se pueden utilizar con `lines()`, `points()` y `abline()`.

3.1. Función `abline()`

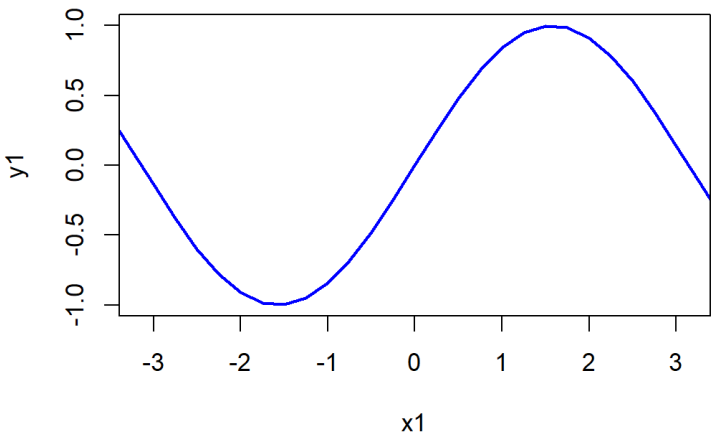
La función `abline()` tiene tres modos básicos:

1. `abline(a,b)`: dibuja la recta $y = a + bx$.
2. `abline(h=y)`: dibuja la recta horizontal en `y`.
3. `abline(v=x)`: dibuja la recta vertical en `x`.

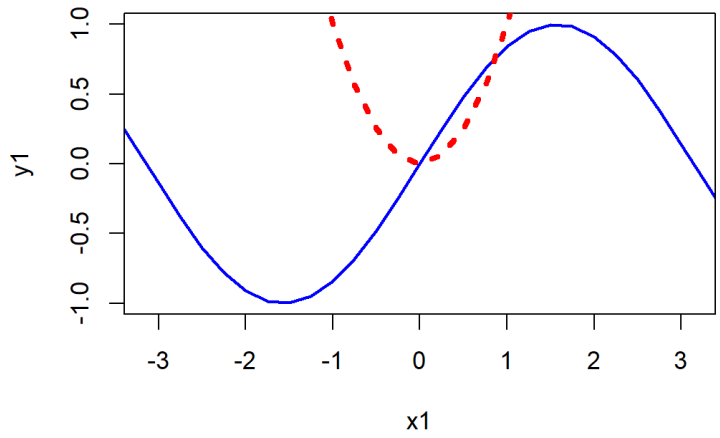
3.2. Un ejemplo

Observa como al ir ejecutando los siguientes comandos se va construyendo una gráfica con muchos elementos.

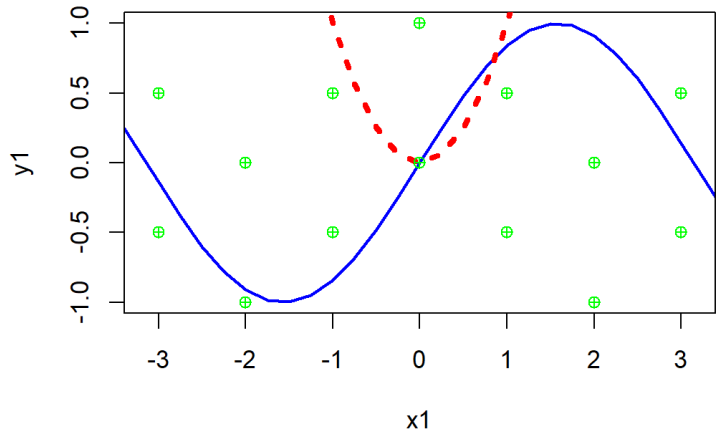
```
x1 <- seq(-4, 4, by=0.25)
y1 <- sin(x1)
plot(x1, y1, type = "l", col = "blue", lwd = 2, xlim = c(-pi,pi), ylim
    = c(-1,1))
```



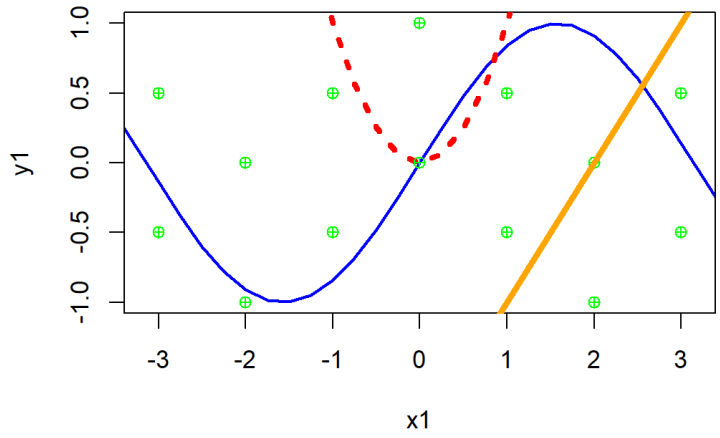

```
x2 <- seq(-4, 4, by=0.25)
y2 <- x2^2
lines(x2, y2, col = "red", lty = 3, lwd = 4)
```



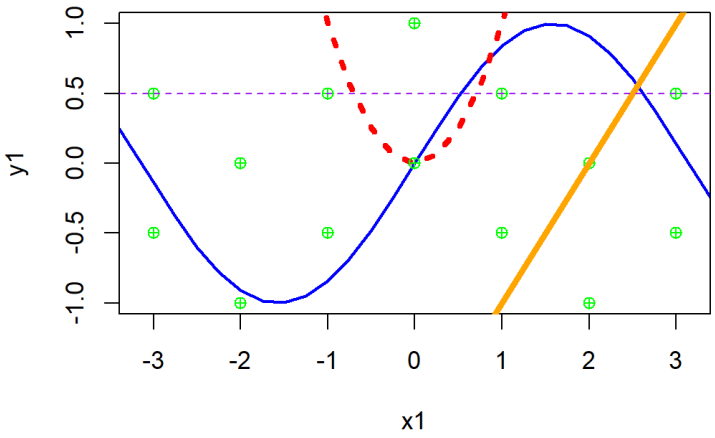

```
x3 <- c(-3,-3,-2,-2,-1,-1,0,0,1,1,2,2,3,3)
y3 <- c(0.5,-0.5,-1,0,0.5,-0.5,0,1,0.5,-0.5,-1,0,0.5,-0.5)
points(x3, y3, col = "green", pch = 10)
```



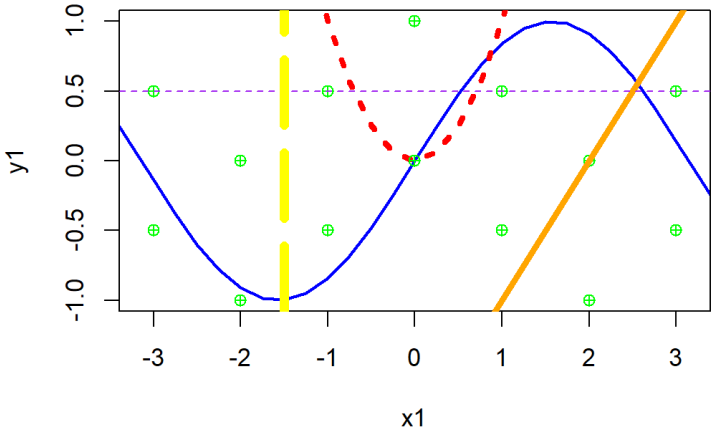

```
abline(-2,1, col = "orange", lwd = 4)
```



```r
abline(h = 0.5, col = "purple", lty = 2)
```




```r
abline(v = -1.5, col = "yellow", lwd = 6, lty = 5)
```



3.3. Leyenda

Podemos añadir una leyenda con la función `legend()`. Algunos de sus argumentos son los siguientes:

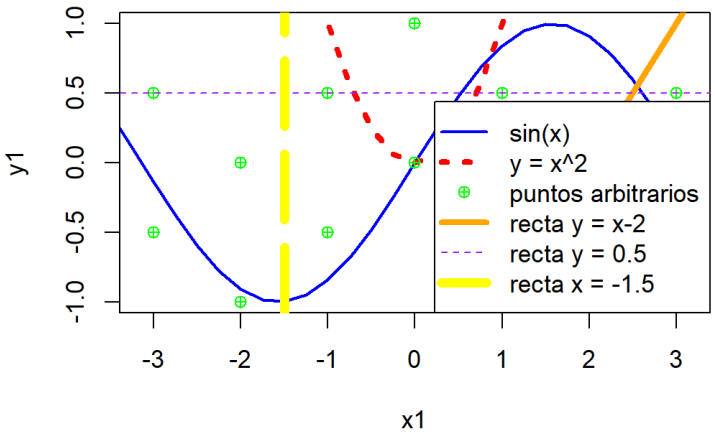
1. Podemos especificar las coordenadas `x,y` donde queremos que aparezca la leyenda. También podemos usar una posición predefinida como las que se enumeran a continuación:

   `bottom`, `left`, `right`, `top`, `bottomleft`, `bottomright`, `topleft`, `topright`.

2. `legend`: un vector que contiene las etiquetas que describen los elementos del gráfico.

3. `col`, `pch`, `lty`, `lwd`: vectores que contienen las características del trazo de cada elemento. Cuando no necesitemos especificar nada, usaremos `NA`.

```
legend("bottomright",
       legend = c("sin(x)", "y = x^2", "puntos arbitrarios",
                  "recta y = x-2", "recta y = 0.5", "recta x = -1.5"),
       col = c("blue","red","green","orange","purple","yellow"),
       pch = c(NA, NA, 10, NA, NA, NA),
       lty = c(1,3,NA,1,2,5),
       lwd = c(2,4,NA,4,1,6))
```

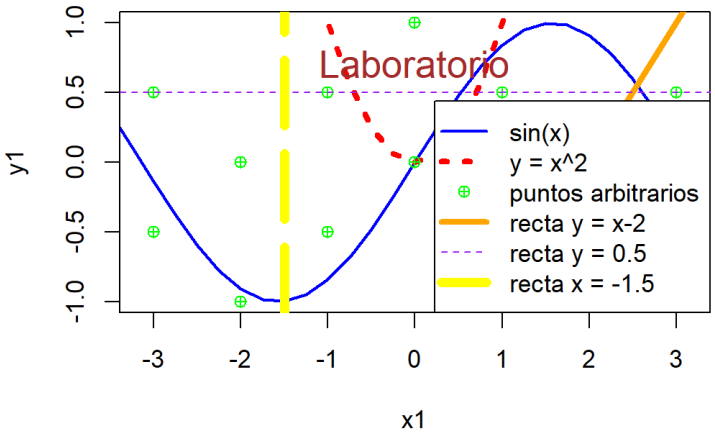


3.4. Texto

La función `text()` nos permite añadir texto. Sus principales argumentos son:

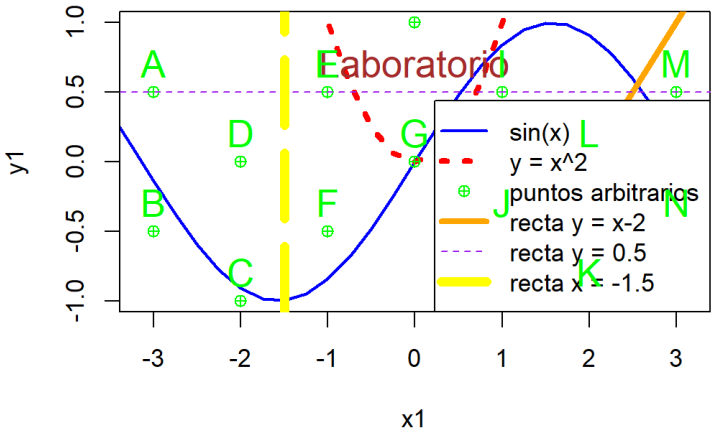
1. Podemos especificar las coordenadas `x,y` donde queremos que aparezca el texto.
2. `labels`: contiene el texto a mostrar.
3. `pos`: posición del texto: `1`: abajo, `2`: izquierda, `3`: arriba, `4`: derecha.
4. `cex`, `col`: tamaño y color del texto.

```
text(0,0.5, labels = "Laboratorio", pos = 3,  col = "brown", cex = 1.5)
```



También podemos etiquetar varios puntos al mismo tiempo. Por ejemplo, podemos querer etiquetar los puntos verdes con distintos nombres. En este caso, `labels` recibiría un vector como argumento.

```
text(x3, y3,
   labels = c("A","B","C","D","E","F","G","H","I","J","K","L","M","N"),
   pos = 3,
   col = "green",
   cex = 1.5)
```



Ejercicio 2. Representa las gráficas de las siguientes funciones:

$$y = 1, \qquad y = \ln(x), \qquad y = \sqrt{x}, \qquad y = x, \qquad y = x\ln(x),$$
$$y = x^3, \qquad y = x^3, \qquad y = 2^x, \qquad y = x^x,$$

en una única gráfica con R. El título será `Complejidades`. Cada función deberá tener un color. Deberás representar únicamente la parte positiva de los ejes x e y, esto es, deberás usar `xlim = c(0,a)` y `xlim = c(0,b)`, siendo `a` y `b` los valores que consideres adecuados. Además, deberás incluir una leyenda que describa todas las gráficas. Las etiquetas que usarás deberán ser de la forma `O(funcion)`. Por ejemplo, `O(x·ln(x))` para $y = x\ln(x)$ u `O(2^x)` para $y = 2^x$.

4. Múltiples gráficas

La función `par(mfrow)` nos permite construir un gráfico compuesto de varios subgráficos. El argumento `mfrow` es un vector de dos componentes que especifica la cantidad de filas y de columnas que tendrá la gráfica. Cada vez que se ejecuta una función plot, se rellenará el siguiente subgráfico. Observa el ejemplo siguiente:

```
par(mfrow = c(2, 3))

x <- seq(-15, 15, by=0.01)
y1 <- sin(x)
plot(x,y1,type = "l", col = "blue", main = "sin(x)",
     lwd = 2, xlim = c(-2*pi,2*pi), ylim = c(-1,1))

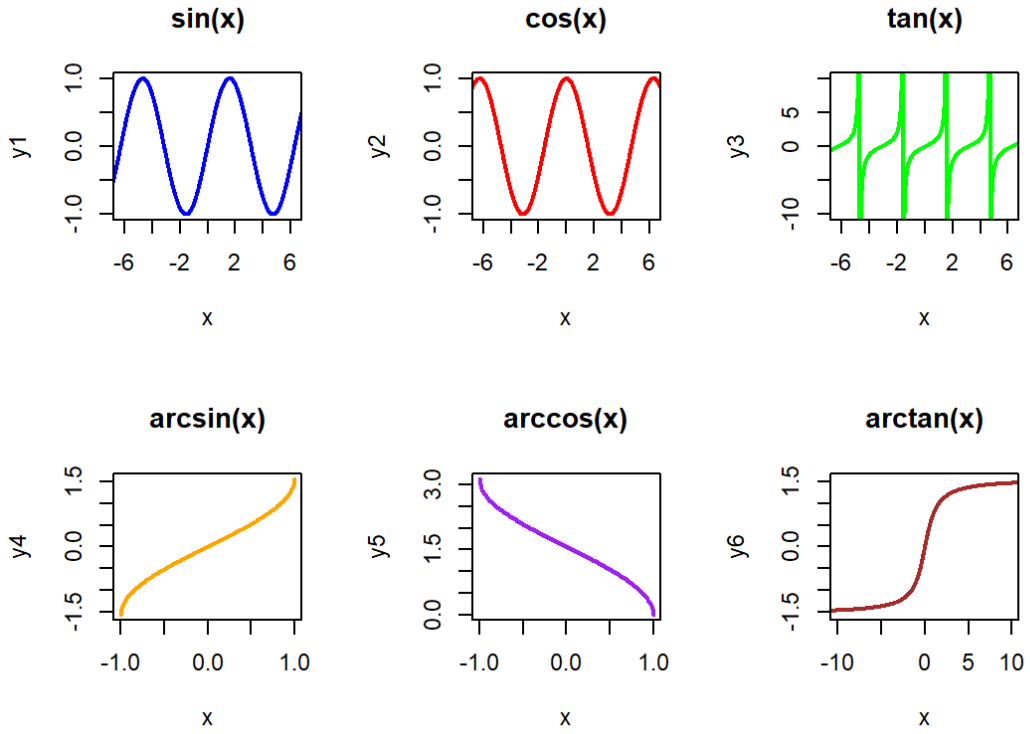
y2 <- cos(x)
plot(x,y2,type = "l", col = "red", main = "cos(x)",
     lwd = 2, xlim = c(-2*pi,2*pi), ylim = c(-1,1))
```

```r
y3 <- tan(x)
plot(x,y3,type = "l", col = "green", main = "tan(x)",
     lwd = 2, xlim = c(-2*pi,2*pi), ylim = c(-10,10))

y4 <- asin(x)
plot(x,y4,type = "l", col = "orange", main = "arcsin(x)",
     lwd = 2, xlim = c(-1,1), ylim = c(-pi/2,pi/2))

y5 <- acos(x)
plot(x,y5,type = "l", col = "purple", main = "arccos(x)",
     lwd = 2, xlim = c(-1,1), ylim = c(0,pi))

y6 <- atan(x)
plot(x,y6,type = "l", col = "brown", main = "arctan(x)",
     lwd = 2, xlim = c(-10,10), ylim = c(-pi/2,pi/2))
```



Ejercicio 3. Debes crear las siguientes gráficas múltiples a partir de las gráficas que has generado en el ejercicio 1:

1. **Funciones polinomiales y raíces:** puedes omitir la gráfica del polinomio con exponente $n = 6$.
2. **Funciones exponenciales y logaritmos:** añade la gráfica de la función $f(x) = 2^x$ que has generado en el ejercicio anterior.
3. **Funciones trigonométricas:** no te limites a copiar el ejemplo. Usa los mismos estilos que has usado en el ejercicio 1.
4. **Funciones trigonométricas hiperbólicas:** aquellas que acaban en `h`: `sinh`, `cosh`, etc.

5. **Otras funciones:** el resto de funciones que has representado en el ejercicio 1 y que no están contempladas en el apartado anterior.

5. Documentación del código

El código debe estar correctamente organizado y comentado. El objetivo no es únicamente conseguir que los gráficos se generen correctamente, sino escribir código fácil de leer y modificar.

Ejercicio 4. Crea un fichero `README.md` para documentar esta práctica. El aspecto deberá ser similar al que encontrarás en la página siguiente.

Entrega

Sube los ficheros con los ejercicios a tu GitHub en la fecha acordada con el profesor. Para esta práctica:

1. guarda tus scripts en la carpeta de la práctica correspondiente;
2. utiliza nombres de fichero claros y consistentes;
3. comenta el código para facilitar su comprensión;
4. comprueba que puedes ejecutar de nuevo cada script desde el principio;
5. evita depender de instrucciones que hayan quedado solamente en la consola

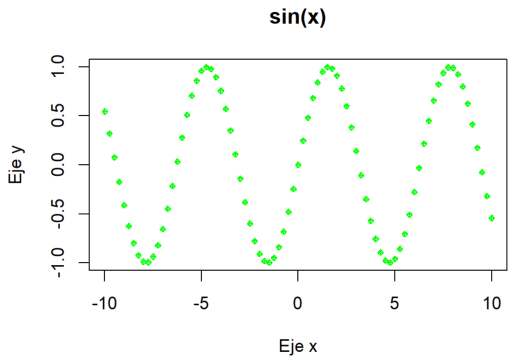
Prácticas de Laboratorio de Matemáticas

Práctica 7: Gráficos en R

El objetivo de esta práctica es introducir la generación de gráficos en R.

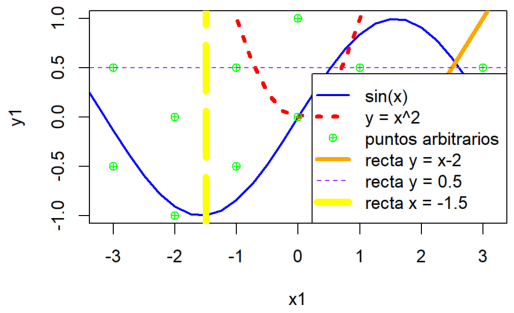
Ejercicio 1: Gráficos básicos en R

El objetivo de este ejercicio es el de aprender a utilizar la función `plot()` de R para dibujar gráficos simples. Hemos representado las gráficas de algunas de las funciones matemáticas más importantes, cada una de ellas con un estilo diferente. Las gráficas se encuentran en el directorio `exercise_1`. Algunos ejemplos son los siguientes: ***(introduce al menos 3 gráficas de las que has generado tú)***



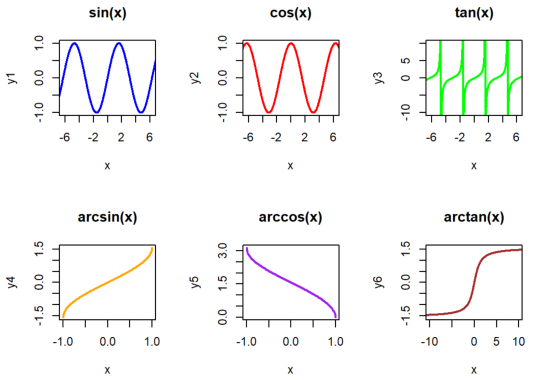
Ejercicio 2: Gráficos en R con varios elementos

El objetivo de este ejercicio es el de aprender a utilizar las funciones `lines()`, `points()`, `abline()` y `legend()`. Como resultado de este ejercicio, hemos generado un gráfico, que se encuentra en el directorio `exercise_2`, que representa las complejidades básicas: ***(introduce tu gráfico)***



Ejercicio 3: Gráficos y subgráficos en R

En objetivo de este ejercicio es el de aprender a utilizar la función `par(mfrow)`. Para ello, hemos agrupado as funciones básicas trabajadas en el tema 1 en varios grupos. Para cada grupo, hemos representado una gráfica compuesta por distintas subgráficas, cada una de ellas representando una de las funciones de ese grupo. El resultado se encuentra en `exercise_3`. El resultado es el siguiente: ***(añadir todas las gráficas de este ejercicio)***



Lista de ejercicios completados

- [x] **Ejercicio 1:** Se encuentra en `ej1.R`. En este ejercicio hemos realizado...
- [x] **Ejercicio 2:** Se encuentra en `ej2.R`. En este ejercicio hemos realizado...
- [ ] **Ejercicio 3:** Se encuentra en `ej3.R`. En este ejercicio hemos realizado...

Información sobre esta práctica

Autor: Miguel Beltrá Vidal.
Correo: miguel.beltravidal@uchceu.es
Asignatura: Laboratorio de Matemáticas.
Institución: CEU Cardenal Herrera.
Estudios: Grado en Ingeniería Matemática/Doble Grado en Física e Ingeniería Matemática.
Curso: 2026-27.